

Системный аудит и corrective-план для «Product Factory» в соло-режиме с агентами

Исполнительное резюме

Вы строите не «платформу разработки» в классическом смысле, а **детерминированный engine поставки артефактов** (execution core) с **контролируемым недетерминированным интеллект-слоем** (агенты/LLM). Это направление стратегически сильное: оно согласуется с тем, как современные поставщики агентных платформ рекомендуют строить наблюдаемую, оцениваемую и управляемую агентность (structured outputs, approvals, trace/evals). ¹

Главный практический риск, который вы описали, подтверждается исследованием первоисточников: **если “ядро выполнения” не формализовано и не воспроизводимо, агентный слой неизбежно превращается в prompt-хаос**, потому что (а) внешние действия неидемпотентны и ретраятся, (б) policy enforcement должен быть отделён от логики, (в) supply-chain артефакты должны формироваться в доверенной контрольной плоскости, а не «тенантом» (т.е. тем, кто пишет код/шаги). Это прямо поддерживается публикациями по workflow-исполнению (идемпотентные activity с ретраями), policy-движкам (PDP/PEP), supply-chain (SLSA provenance/attestations) и SBOM-практикам. ²

Ниже — строгое разложение ответственности слоёв, blueprint минимального execution-ядра, затем дорожная карта (Small/Medium/Large для соло-режима), риски, KPI/мониторинг, governance/change-control и шаблоны артефактов.

Таблица допущений

Параметр	Принятое допущение	Почему
Роль	Я — соло-разработчик, использую агентов как ускоритель, но оставляю контроль side-effects за детерминированным контуром	Это соответствует best practice “keep approvals on” и разделению tool-calling vs execution ³
Старт	2026-02-20	Указано вами
Домен продукта	Не указан; первым archetype считаю «API-сервис»	Это самый проверяемый артефакт для фабрики (build/test/deploy) и минимальный риск
Окружения	dev + staging (prod позже)	Для снижения риска: promotion только после gates (тесты+security+eval) ⁴

Параметр	Принятое допущение	Почему
Оркестрация	Сначала “минимальный orchestrator” (внутренний), затем durable engine (Temporal/Argo) по триггерам	Durable engine добавляет устойчивость и историю (audit), но увеличивает ops-нагрузку; activity должны быть идемпотентны ⁵

Проверка тезисов и уточнение «минимального ядра»

Почему тезис «сначала execution core без AI» технически оправдан

Ваш тезис «execution engine без AI — уже продукт» подтверждается тем, как устроены надёжные системы оркестрации:

- В durable workflow-подходе внешние/недетерминированные операции изолируются в **activities**, которые **автоматически ретраятся** и **должны быть идемпотентны**, иначе повторы приводят к дублям и побочным эффектам. ⁶
- В Kubernetes-ориентированном workflow-движке ретраи и политики retryStrategy — нормальная часть модели, потому что шаги могут падать по причинам «вне вашего кода» (ошибки/удаления/сбои), и система должна уметь повторять выполнение. ⁷

Из этого следует жесткое правило архитектуры фабрики:

Любая операция, которую “можно повторить”, обязана быть идемпотентной, а контроль повторов обязан жить в execution-ядре.

Агент никогда не должен быть “механизмом повторов” — он слишком дорогой и недетерминированный.

Это же логически согласуется с supply-chain требованиями: SLSA требует, чтобы provenance/attestations генерировались в доверенной контрольной плоскости (control plane) и не могли быть подделаны шагами, которые контролирует “тенант” (условно: пользовательские build-скрипты). ⁸

Почему «формализованный input contract» — не косметика, а безопасность

Ваш тезис о необходимости `product.yaml / constraints.yaml / quality_profile.yaml / risk_profile.yaml / target_stack.yaml` — это фактически перевод agent-взаимодействия в **structured outputs + строгие схемы**, что является базовой рекомендацией для уменьшения риска prompt injection и неконтролируемой агентности (excessive agency). ⁹

Ключевые подтверждения из первоисточников:

- OpenAI-гайд по tool/function calling описывает flow как многошаговый протокол, где **ваше приложение исполняет tool**, а модель только запрашивает вызов; также рекомендуется включать **strict mode**, чтобы вызовы соответствовали схеме. ¹⁰
- OWASP Top 10 for LLM Applications прямо выделяет Prompt Injection и Excessive Agency как ключевые риски. ¹¹

- OpenAI “Safety in building agents” рекомендует: не смешивать недоверенные данные с управляющими инструкциями, включать approvals, использовать guardrails и trace evals/ graders. ¹²

Где нужно усилить ваш текст-аудит

Ваш аудит уже правильно расставляет акценты. Две добавки, которые существенно повышают инженерную жёсткость:

- 1) **Определить не только слои, но и “инварианты” между слоями:** какие данные могут пересекать границу, в каком формате, и какие проверки обязаны быть пройдены. Это напрямую соответствует подходу NIST SSDF: безопасность «встраивается» в жизненный цикл, а не добавляется в конце. ¹³
- 2) **Сделать supply-chain и “auditability” частью execution-core,** а не отдельным этапом позже: CISA в обновлении SBOM подчёркивает необходимость частого обновления SBOM по версиям/релизам, покрытие транзитивных зависимостей и операционные практики доставки/обновлений SBOM. ¹⁴

Архитектурный blueprint v1

Слои и границы ответственности

Ниже — модель ответственности, которая делает вашу формулу “AI генерирует proposals, core валидирует и применяет” реализуемой без расползания.

Слой	Ответственность	Что строго запрещено	Главные артефакты
Input Formalization	Превратить намерение в формализованный контракт проекта/ запуска	Любые side-effects	product.yaml, constraints.yaml, quality_profile.yaml, risk_profile.yaml, target_stack.yaml
Deterministic Execution Core	Оркестрация шагов, ретраи, идемпотентность, аудит, артефакты, policy enforcement	“Умные решения” на основании текста без схемы	Event log, State machine, Artifact registry, Policy decisions
Tool Executor (sandbox)	Исполнение side-effects как изолированных инструментов	Прямое выполнение free-form команд от модели	Tool contracts (JSON schema), idempotency keys, audit trail

Слой	Ответственность	Что строго запрещено	Главные артефакты
Agent Runtime	Генерация предложений (diff/ADR/тесты/планы), анализ	Прямой доступ к секретам/деплою/базам без approvals	Proposals: patch sets, ADR drafts, test plans
Memory / Knowledge	Версионированные источники истины, индексы, решения	“Память” без provenance/версий	Decision log, index versions, datasets
Observability / Evals	Измерения, trace grading, регрессионные тесты агента	Продвижение без gates	Traces, eval reports, dashboards

Эта модель прямо опирается на: разделение PDP/PEP (OPA), необходимость audit trail для решений политик, и практики trace grading/evals как инструмент воспроизводимых оценок агентного поведения. ¹⁵

Архитектура v1 в Mermaid

```

flowchart TB
    subgraph INPUT[Input Contract]
        P[product.yaml]
        C[constraints.yaml]
        Q[quality_profile.yaml]
        R[risk_profile.yaml]
        T[target_stack.yaml]
    end

    subgraph CORE[Deterministic Execution Core]
        API[Factory API]
        SM[State machine + durable history]
        LOG[(Event log / audit)]
        POL[Policy PDP (OPA/Rego)]
        ART[(Artifact registry + versions)]
    end

    subgraph EXEC[Tool Execution]
        REG[Tool registry (JSON schemas)]
        EXE[Sandbox executor + idempotency + retries]
    end

    subgraph AI[Agent Proposal Layer]
        AR[Agent runtime]
        LLM[LLM API]
        GR[Trace graders + eval runs]
    end

```

```

end

subgraph OBS[Observability]
  OTel[OpenTelemetry + OTLP]
  M[(Metrics)]
  L[(Logs)]
  Tr[(Traces)]
end

INPUT --> API --> SM --> REG --> EXE
SM --> POL
EXE --> ART
SM --> LOG
EXE --> LOG

AR --> LLM
AR --> REG
GR --> AR
SM --> GR

API --> OTel
SM --> OTel
EXE --> OTel
AR --> OTel
OTel --> M
OTel --> L
OTel --> Tr

```

Почему именно так:

- ОРА определяет паттерн PDP/PEP: решения политики отделены от enforcement и могут логироваться для аудита и отладки. ¹⁶
- OTLP как протокол телеметрии стабилен для traces/metrics/logs и описывает транспорт/доставку данных между источниками, коллекторами и backend'ами — это снижает lock-in на наблюдаемость. ¹⁷
- Trace grading концептуально требует “полного trace того, что произошло”, и даёт воспроизводимые оценки на уровне шагов/вызовов/tools. ¹⁸

Формализация input contract v1

Нижe — **полностью заполненные** примеры входных файлов для одного типа продукта: `catalog-service` (HTTP API + PostgreSQL). Они предназначены для: (а) валидации схемой, (б) генерации кода/archetype, (в) строгих gates.

```

# product.yaml
apiVersion: productfactory.io/v1

```

```
kind: ProductSpec
metadata:
  productId: catalog-service
  owner: solo-dev
  createdAt: 2026-02-20
product:
  type: api-service
  name: catalog-service
  description: "HTTP API для каталога товаров: CRUD + поиск"
  interfaces:
    http:
      basePath: /api/v1
      auth: bearer-jwt
data:
  stores:
    - name: catalog-db
      kind: postgres
      schema: catalog
nonFunctional:
  availabilityTarget: "99.5"
  maxP95LatencyMs: 250
  maxErrorRatePercent: 0.5
```

```
# constraints.yaml
apiVersion: productfactory.io/v1
kind: Constraints
security:
  secrets:
    storage: env-and-vault
    loggingRedaction: true
  dataClassification:
    pii: none
    retentionDays: 30
    allowedOutboundDomains:
      - api.openai.com
      - github.com
platform:
  runtime: docker-compose
  os: linux-amd64
  containerRegistry: ghcr.io
delivery:
  ci: github-actions
  cd: gitops-staging
```

```

# quality_profile.yaml
apiVersion: productfactory.io/v1
kind: QualityProfile
testing:
  unitCoverageMinPercent: 70
  integrationTestsRequired: true
  smokeTestsRequired: true
securityGates:
  sbom:
    format: cyclonedx-json
    required: true
  vulnerabilityScan:
    tool: trivy
    failOnSeverity:
      - CRITICAL
      - HIGH
  signing:
    tool: cosign
    required: true
observability:
  tracing:
    required: true
    protocol: otlp-http
  metrics:
    required: true

```

```

# risk_profile.yaml
apiVersion: productfactory.io/v1
kind: RiskProfile
riskTier: medium
agentPolicy:
  approvalsRequiredFor:
    - write_repository
    - deploy_staging
    - create_secrets
  budgets:
    maxLlmTokensPerRun: 250000
    maxToolCallsPerRun: 40
    maxWallClockSecondsPerRun: 900
threatModel:
  llmTop10Focus:
    - prompt_injection
    - insecure_output_handling
    - excessive_agency

```

- model_dos
- supply_chain

```
# target_stack.yaml
apiVersion: productfactory.io/v1
kind: TargetStack
language:
  name: kotlin
  framework: ktor
packaging:
  container: docker
database:
  primary: postgres
observability:
  tracing: opentelemetry
  metrics: prometheus
```

Почему это снижает риск: агенту проще “предложить изменение”, чем “изобрести систему”, а строгие бюджеты/approvals соответствуют рекомендациям держать tool approvals включенными и не допускать, чтобы недоверенные данные напрямую управляли действиями. ¹⁹

Roadmap и план работ для соло-режима

Три варианта масштаба (Small / Medium / Large)

Вариант	Цель	Срок (реалистично для соло)	Результат
Small	Один archetype, один pipeline, staging-деплой, минимальные policies/аудит	8–12 недель	Воспроизводимый “ProductSpec → Artifacts → Staging”
Medium	2–3 archetype + eval-ворота + SBOM/подпись + GitOps promotion	6–12 месяцев	Фабрика как продукт (измеримая, управляемая, расширяемая)
Large	Multi-tenancy + портал self-serve + расширенный комплаенс	12–18+ месяцев	Программа, а не MVP; соло-риски очень высокие

Соло-ресурсная модель (FTE как «шляпы»)

«Шляпа»	Доля 1.0 FTE в Medium	Что оптимально делегировать агентам	Что оставлять человеку
Архитектура/ SDD/ADR	0.15	черновики ADR, сравнение опций	финальные границы слоёв и decision points
Execution core	0.25	генерация тестов/fixtures	идемпотентность, ретрай, семантика state machine ⁶
Policy & security	0.15	чек-листы OWASP, черновой threat mapping	политики доступа, секреты, подписывание ²⁰
Agent layer	0.15	генерация proposals (diff/ADR/tests)	контроль tool contracts, budgets/approvals ²¹
CI/CD & supply chain	0.15	YAML pipelines, отчёты	правила fail/pass, подписи/проверка, SBOM ²²
Observability & evals	0.15	дашборды, baseline-наборы	метрики, SLO, интерпретация regressions ²³

Фазы и deliverables (как чек-листы)

Фаза Alpha — Input Contract Spec v1 (2 недели)

- Deliverables: - [] Определены и зафиксированы версии схем для пяти входных файлов (как выше)
- [] Валидатор контрактов (CLI команды `pf validate`) возвращает детерминированные ошибки
- [] Минимальный каталог risk tiers: `low/medium/high` и мэппинг approvals/budgets
- [] ADR-0001: «границы слоёв + запреты» (см. шаблон ниже)

Критерии успеха (измеримые): - Вся конфигурация фабрики проходит валидацию на до-запуске; “free-form” поля ограничены и не участвуют в policy decisions напрямую (подход “extract only structured fields”). ²¹

Фаза Beta — Minimal Deterministic Execution Core (6 недель)

- Deliverables: - [] State machine: `NEW → PLANNED → GENERATED → TESTED → SECURED → STAGED → DONE/FAILED`
- [] Event log: каждое событие имеет `runId`, `stepId`, `inputDigest`, `outputDigest`
- [] Tool registry (JSON schema) и sandbox executor с идемпотентными ключами
- [] Policy PDP (OPA) с решениями “allow/deny” и decision logs
- [] Артефакт-реестр: версии репо/образа/SBOM/подписи связаны в один “run record”

Критерии успеха: - При падении шага ретрай не создаёт двойных side-effects (идемпотентность) — это базовый принцип activities/ретраев. ⁵

- Policy decisions логируются для аудита и offline debugging. ²⁴

Фаза MVP — Factory Run для одного archetype (6–8 недель)

Deliverables: - [] Archetype `catalog-service` (код + тесты + Docker + миграции БД)

- [] CI: unit + integration + сборка образа
- [] SBOM генерация (Syft)
- [] Vulnerability scan (Trivy)
- [] Подпись образа (Cosign) + verify в gate
- [] Staging deploy (GitOps) + smoke test

Источники, которые обосновывают состав gates: - Syft описывает генерацию SBOM для контейнеров/файлов и поддерживаемые форматы (CycloneDX/SPDX). ²⁵

- Trivy описывает модель severity и источники advisories, что важно для политики “fail on HIGH/CRITICAL”. ²⁶

- Cosign описывает процедуру `cosign verify` и проверки digest/claims. ²⁷

- CISA SBOM guidance подчёркивает частоту обновления SBOM по версиям/релизам, покрытие зависимостей и практики доставки/обновления SBOM. ¹⁴

Критерии успеха: - 3 последовательных прогона “один и тот же ProductSpec” дают одинаковый set артефактов (по digest) и воспроизводимые отчёты gates.

- Любой staging-деплой можно откатить revert’ом desired state (если используете GitOps). При этом учтите ограничения autosync/rollback в Argo CD (rollback невозможен при включённом autosync). ²⁸

Фаза Scale — AI как Proposal Engine (8–10 недель)

Deliverables: - [] Агент “планировщик” генерирует: pipeline plan, ADR draft, test plan как JSON-структуры

- [] Агент “кодоген” генерирует patch-наборы (git diff) без прямого merge
- [] Approvals включены для write/deploy операций (human-in-the-loop)
- [] Trace grading и eval runs для регрессионной оценки агентных изменений

Обоснование: - OpenAI рекомендует держать tool approvals включенными, использовать guardrails и trace graders/evals, а также проектировать flow так, чтобы недоверенные данные не управляли tool calls напрямую. ¹²

- Function calling в строгом режиме повышает соответствие схеме и уменьшает риск некорректных tool-вызовов. ²⁹

Критерии успеха: - Любое изменение prompt/tools/policy автоматически прогоняет eval suite и блокирует регресс. ¹⁸

Gantt-таймлайн (старт 2026-02-20)

```
gantt
  title Product Factory (solo): roadmap от execution-core к proposal-agents
  dateFormat YYYY-MM-DD

  section Alpha
  Input Contract Spec v1 + валидатор :a1, 2026-02-20, 14d
```

section Beta	
Minimal Execution Core (SM+EventLog+Tools)	:b1, 2026-03-06, 42d
Policy PDP (OPA) + decision logs	:b2, 2026-03-13, 35d
Observability baseline (OTel/OTLP)	:b3, 2026-03-13, 35d
section MVP	
Archetype catalog-service + CI build/tests	:c1, 2026-04-17, 28d
SBOM (Syft) + scan (Trivy) + sign (Cosign)	:c2, 2026-04-24, 28d
GitOps staging deploy + smoke tests	:c3, 2026-05-01, 28d
section Scale	
Agent Proposal Engine (diff/ADR/tests)	:d1, 2026-05-29, 42d
Trace grading + eval runs + regression gates	:d2, 2026-06-05, 56d
section Medium+	
Второй archetype (web или data pipeline)	:e1, 2026-08-01, 60d
Supply-chain maturity (SLSA targets, VSA)	:e2, 2026-08-15, 90d
Multi-tenancy/self-serve (опционально)	:e3, 2026-11-01, 120d

Реестр рисков и меры контроля

Ниже — risk register, ориентированный на соло-режим и ваш ключевой риск “размытая ответственность слоёв”.

Риск	Вероятность	Влияние	Митигирующие меры	План на случай сбоя
Размытая граница слоёв (AI начинает “управлять”)	Высокая	Высокое	Жёсткий input contract + strict schemas; approvals на write/deploy; PDP/PEP с audit	Отключить privileged tools; перевести агента в read-only; расследование по trace logs ³⁰
Unbounded consumption / Model DoS	Высокая	Высокое	Budgets: tokens/tool calls/wall-time; политика отключения параллельных tool calls при необходимости	Kill workflow по бюджетам; пост-анализ; корректировка policy ³¹
Prompt injection (через RAG/ внешний текст)	Средняя	Высокое	“Untrusted data never drives tools”; структурная экстракция; guardrails; approvals	Отключить внешние источники; регресс-фикс; ротация ключей при утечках ¹⁹

Риск	Вероятность	Влияние	Митигирующие меры	План на случай сбоя
Дубли side-effects из-за ретраев	Средняя	Высокое	Идемпотентность activities/tools; idempotency keys; тесты ретраев	Cleanup workflow; откат desired state; фиксы и регрессионные тесты ⁵
Supply-chain компрометация	Средняя	Высокое	SBOM+scan+sign+verify gates; provenance/ attestation политика	Откат на последний подписанный digest; блок релиза; ускоренный патч ³²
Слом rollback из-за GitOps настроек	Средняя	Среднее/ Высокое	Учитывать ограничения Argo CD autosync (rollback невозможен при enabled autosync) и фиксировать процедуру	Временно отключить autosync, выполнить revert, затем включить обратно ²⁸
Регуляторные требования по AI	Средняя	Среднее	Минимальный governance по AI RMF/ ISO 42001; watch-процесс; decision points	Переприоритизация roadmap; ограничение функций, попадающих в high-risk категорию ³³
Масштабирование инфраструктуры (Kubernetes “слишком рано”)	Средняя	Среднее	Начать с простого runtime; переходить к K8s по триггерам	Миграция на managed-K8s после стабилизации core; избегать “kubeadm сразу” ³⁴

Контекст по регуляторике (если вы в ЕС/работаете с ЕС-рынком): AI Act применяется поэтапно; большинство правил начинает применяться с 2 августа 2026, а отдельные требования — раньше/ позже. ³⁵

KPI, мониторинг, governance и change-control

KPI: что мерить, чтобы фабрика не стала “идеей”

Delivery-метрики (DORA-классика) нужны, чтобы видеть скорость и стабильность поставки. ¹⁸

Agent-метрики нужны, чтобы управлять качеством/стоимостью agent-слоя.

KPI	Как измерять	Почему важно
Cycle time: spec → staged	timestamps событий state machine	Главная метрика фабрики: скорость превращения контракта в артефакт
Gate pass rate	% прогонов, прошедших test+scan+sign+eval	Иначе вы масштабируете регрессии, а не продуктивность ³⁶
Cost per run (LLM)	токены×цены / runId	Контроль unbounded consumption и бюджета ³⁷
Tool retries / side-effect incidents	события ретраев + инциденты дублей	Показывает качество идемпотентности tools/activities ⁵
Trace-graded quality	scorecards по trace grading	Нужны воспроизводимые оценки агентного поведения ¹⁸
Policy denies	OPA decision logs	Проверка, что governance реально работает, а не декларативно ³⁸

Наблюдаемость: почему OTLP/OTel — “мета-контракт”

OTLP определяет кодирование/транспорт/доставку телеметрии между источниками, коллекторами и backend'ами и объявлен stable для traces/metrics/logs. Это делает его хорошим “нейтральным” протоколом для фабрики, где вы не хотите привязываться к одному поставщику наблюдаемости на старте. ³⁹

Governance: decision points и ADR-триггеры

Рекомендуемый набор decision points (фиксируйте как ADR-пересмотр по метрикам):

- **LLM API → self-host**: триггер, когда cost per run становится доминирующим, или появляются требования суверенности данных (персональные/чувствительные). (Цены по токенам должны быть источником расчёта.) ⁴⁰
- **Простая оркестрация → durable engine**: триггер, когда начинают возникать длинные прогоны, необходимость pause/resume и сложные компенсации; activity должны быть идемпотентны. ⁶
- **pgvector → выделенная vector DB**: триггер по latency/фильтрации/объёму; (выбирайте поздно, чтобы не платить ops-налог заранее).
- **Добавление нового tool с side-effects**: всегда high-risk change → threat mapping (OWASP/NIST) + policy + approval. ⁴¹

Change-control: три класса изменений

- **Low-risk**: docs/refactor без изменения схем, policies, tools.
- **Medium-risk**: новый archetype, изменения CI, изменения quality thresholds.
- **High-risk**: новый tool с side-effects, изменение budget/approval/policy, секреты, автопромоушен.

С практической стороны (репозиторий): “security hardening” для GitHub Actions рекомендует снижать риск секретов, использовать OIDC вместо long-lived credentials (где применимо) и помнить, что

компрометация одного action может быть критичной. 42

Это напрямую связано с тем, что ваш CI/CD — часть execution-core.

Бюджет, rollout/rollback, инструменты и шаблоны

Бюджет: три уровня детализации + альтернативные сценарии

Низкая детализация (order-of-magnitude)

Вариант	Срок	Главные драйверы стоимости
Small	2–3 месяца	ваше время + умеренные LLM-запуски 43
Medium	6–12 месяцев	LLM + хранение артефактов/логов/трейсов + CI runners 44
Large	12–18+ месяцев	multi-tenancy, compliance, ops-налог, SLO-эксплуатация 45

Средняя детализация (структура расходов)

- **LLM токены:** считаются по официальным ценам за 1М токенов (input/cached/output). 43
- **Evals/trace grading:** дополнительные прогоны и хранение трасс; trace grading — инструмент для масштабного выявления ошибок. 18
- **Supply chain tooling:** Syft/Trivy/Cosign — OSS-инструменты, но требуют времени интеграции и хранения артефактов (SBOM/attestations). 22
- **Наблюдаемость:** OTLP/OTel снижает lock-in, но backend и ретеншн стоят денег/ресурсов. 17

Высокая детализация (конкретный расчёт LLM-стоимости на месяц)

Официальный прайс OpenAI на текстовые токены (за 1М):

- gpt-5-mini : input \$0.25, cached input \$0.025, output \$2.00
- gpt-5.1 : input \$1.25, cached input \$0.125, output \$10.00
- gpt-5.2 : input \$1.75, cached input \$0.175, output \$14.00 43

Нижe — **примерные сценарии** (это именно допущения нагрузки; формула и цены — официальные):

Сценарий	Модель	Запусков/день	Input/run	Output/run	Оценка \$/месяц (30д)
Small-dev	gpt-5-mini	10	50k	10k	\$9.75
Medium-factory	gpt-5.1	50	120k	25k	\$600.00
Large-usage	gpt-5.2	200	200k	40k	\$5,460.00

Расчёт логики (пример Medium):

1500 runs * (0.12M*\$1.25 + 0.025M*\$10) = 1500*(0.15 + 0.25) = \$600. 43

Снижение стоимости достигается кэшированием input (цена cached input ниже), уменьшением контекста и разделением задач по моделям. 46

Альтернативные сценарии (по стратегии): - **Lean-local**: минимальная инфраструктура, execution-core локально/на одном сервере; быстрее и дешевле в ops, но сложнее масштабировать.

- **Cloud-first**: быстрее выйти в стабильный staging (managed сервисы), но выше переменные расходы; важно держать budgets и мониторинг. ⁴⁷

- **Hybrid/regulated**: оправдано требованиями суверенности/данных и governance; поддерживается AI RMF/ISO-подходом к системам менеджмента, но увеличивает срок. ⁴⁸

Rollout и rollback: минимально безопасный план

Rollout-принцип: продвигаем только подписанные и прошедшие gates артефакты; rollback должен быть "операцией Git + verify".

- Подпись и проверка: `cosign verify` подтверждает соответствие digest и подписи. ²⁷

- Для GitOps (Argo CD) зафиксируйте в runbook ограничения: autosync имеет семантику, при которой rollback недоступен, если autosync включён; это надо учесть в процедуре. ²⁸

Ниже — пример rollback-процедуры без плейсхолдеров (использованы конкретные имена):

```
# 1) Откат desired state (GitOps):
git checkout main
git revert 3f2a10c1d1c0f7a0d3b2a8a6d1b8c9e6a7f4c0aa
git push origin main

# 2) Проверка подписи последнего "хорошего" образа:
cosign verify ghcr.io/acme/catalog-
service@sha256:97fc222cee7991b5b061d4d4afdb5f3428fcb0c9054e1690313786befa1e4e36

# 3) Smoke test staging:
curl -sS -H "Authorization: Bearer staging-demo-token" http://
staging.acme.internal/api/v1/health
```

Инструменты (минимальный "solo-friendly" набор)

- Policy as code: Open Policy Agent ⁴⁹ как PDP/decision logs. ¹⁶
- Observability: OpenTelemetry + OTLP. ¹⁷
- SBOM: Syft (CycloneDX/SPDX). ²⁵
- Vulnerability scan: Trivy. ²⁶
- Signing/verifying: Sigstore Cosign. ²⁷
- Agent evals: trace grading. ¹⁸
- CI security hardening: GitHub Actions guidance (OIDC, риски third-party actions). ⁴²
- Нормативная опора: AI RMF + ISO/IEC 42001, плюс учёт этапов EU AI Act и GDPR при работе с данными/рынком ЕС. ⁵⁰

Шаблоны (ADR, PR, risk register) — без плейсхолдеров

ADR (шаблон-пример)

ADR-0001: Граница ответственности слоёв и запрет side-effects из agent layer

Дата: 2026-02-20

Статус: accepted

Контекст

Строится Product Factory в соло-режиме. Цель — воспроизводимое создание deployable артефактов.

Риск — "prompt-haos": агентные решения начинают напрямую менять состояние систем.

Решение

- 1) Любые side-effects исполняются только через Tool Executor (sandbox).
- 2) Агентный слой генерирует только proposals (diff/ADR/tests) в структурированном формате.
- 3) Policy PDP принимает решения allow/deny на основании структурного input и контекста запуска.

Инварианты

- Любой tool call имеет JSON schema и strict-валидацию аргументов.
- Любой tool call имеет audit event в event log.
- Любой retry не должен порождать дубли side-effects (идемпотентность).

Триггеры пересмотра

- Инцидент дублирования ресурсов/деплов из-за ретраев
- Рост cost per run > 2x за месяц
- Появление новой категории данных (PII/secret) в контексте запуска

PR template

Цель изменения

Изменение: добавлен tool "deploy_staging" с идемпотентным ключом и policy gate.

Риск-класс

high

Затрагивает

- tool schemas: да
- policies (OPA): да
- side-effects: да
- CI/CD: да
- secrets/доступы: нет

```

## Обязательные проверки (gates)
- [ ] unit tests: PASS
- [ ] integration tests: PASS
- [ ] SBOM generated (Syft): PASS
- [ ] vulnerability scan (Trivy) fail0n: HIGH/CRITICAL: PASS
- [ ] signature verify (Cosign): PASS
- [ ] offline eval suite (trace grading runs): PASS

## План отката
1) git revert коммита с этим PR
2) verify подписи последнего good image digest
3) redeploy staging через GitOps desired state

## Наблюдаемость
- traceId последнего полного прогона: 9b7f1c2d3e4a5f60718293a4b5c6d7e8
- auditRunId: run-2026-02-20-0007

```

Risk register (шаблон)

```

# Risk Register v1 (2026-02-20)

| ID | Риск | Вероятность | Влияние | Владелец | Митигирующие меры |
Контингенс-план | Метрика |
|---|---|---|---|---|---|---|
| R-001 | unbounded consumption | high | high | solo-dev | budgets + policy
denies | kill workflow + read-only mode | cost/run, toolCalls/run |
| R-002 | prompt injection | medium | high | solo-dev | structured extraction +
approvals | disable externals + re-eval | denied tool attempts |
| R-003 | supply-chain compromise | medium | high | solo-dev |
SBOM+scan+sign+verify | rollback to last signed | scan findings |

```

План коммуникаций (даже в соло-режиме)

Стейкхолдер	Канал	Частота	Артефакты
Спонсор/заказчик	демо “spec → staging”	еженедельно	trend cycle time, gate pass rate, cost/run
Безопасность	обзор risk register + SBOM/подпись	раз в 2–4 недели	policy changes, scan отчёты, инциденты
Потребители (разработчики)	README archetype + changelog	по релизу	golden path, ограничения, quickstart

Первичные источники (ссылки в явном виде)

NIST AI RMF 1.0 (PDF): <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>
NIST AI RMF Playbook: <https://airc.nist.gov/airmf-resources/playbook/>
ISO/IEC 42001: <https://www.iso.org/standard/42001>
EU AI Act official text (Regulation (EU) 2024/1689): <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
EU AI Act implementation timeline: <https://ai-act-service-desk.ec.europa.eu/en/ai-act/eu-ai-act-implementation-timeline>
GDPR (Regulation (EU) 2016/679): <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>
OWASP Top 10 for LLM Apps: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
CISA SBOM Minimum Elements (2025): <https://www.cisa.gov/resources-tools/resources/2025-minimum-elements-software-bill-materials-sbom>
SLSA requirements (v1.0): <https://slsa.dev/spec/v1.0/requirements>
OpenTelemetry OTLP spec: <https://opentelemetry.io/docs/specs/otlp/>
OPA (docs): <https://www.openpolicyagent.org/docs/latest/>
Syft (SBOM tool): <https://github.com/anchore/syft>
Trivy vulnerability docs: <https://trivy.dev/docs/v0.52/guide/scanner/vulnerability/>
Cosign verify: <https://docs.sigstore.dev/cosign/verifying/verify/>
OpenAI Agents SDK: <https://platform.openai.com/docs/guides/agents-sdk/>
OpenAI Trace grading: <https://platform.openai.com/docs/guides/trace-grading>
OpenAI Function calling (strict mode): <https://platform.openai.com/docs/guides/function-calling/how-do-i-ensure-the-model-calls-the-correct-function>
OpenAI Pricing: <https://platform.openai.com/docs/pricing/>
GitHub Actions security hardening: <https://docs.github.com/actions/learn-github-actions/security-hardening-for-github-actions>
Kubernetes learning environment note on production-like complexity: <https://kubernetes.io/docs/setup/learning-environment/>

OpenAI ⁵¹ NIST ⁵² ISO ⁵³ European Union ⁵⁴ OWASP ⁵⁵ CISA ⁵⁶ MITRE ⁵⁷ GitHub ⁵⁸

¹ ³ ⁹ ¹² ¹⁹ ²¹ ³⁰ ⁴⁷ Safety in building agents | OpenAI API

https://platform.openai.com/docs/guides/agent-builder-safety?utm_source=chatgpt.com

² ⁵ ⁶ ⁵¹ 102 20240906a

https://learn.temporal.io/assets/files/temporal-102-with-java-replay2025-468b8109d5a9ce33b36c2910c48433d9.pdf?utm_source=chatgpt.com

⁴ ¹⁸ ⁵² Trace grading | OpenAI API

https://platform.openai.com/docs/guides/trace-grading?utm_source=chatgpt.com

⁷ Tolerating Pod Deletion - Argo Workflows - The workflow engine for Kubernetes

https://argo-workflows.readthedocs.io/en/latest/tolerating-pod-deletion/?utm_source=chatgpt.com

8 SLSA • Producing artifacts

https://slsa.dev/spec/v1.0/requirements?utm_source=chatgpt.com

10 29 Function calling - OpenAI API

https://platform.openai.com/docs/guides/function-calling/how-do-i-ensure-the-model-calls-the-correct-function?utm_source=chatgpt.com

11 20 31 41 OWASP Top 10 for Large Language Model Applications | OWASP Foundation

https://owasp.org/www-project-top-10-for-large-language-model-applications/?utm_source=chatgpt.com

13 Secure Software Development Framework | CSRC

https://csrc.nist.gov/projects/ssdf?utm_source=chatgpt.com

14 32 TLP:CLEAR 2025 Minimum Elements for a Software

https://www.cisa.gov/sites/default/files/2025-08/2025_CISA_SBOM_Minimum_Elements.pdf?utm_source=chatgpt.com

15 16 57 How to Deploy OPA | Open Policy Agent

https://www.openpolicyagent.org/docs/deploy?utm_source=chatgpt.com

17 23 39 44 49 OTLP Specification 1.9.0 | OpenTelemetry

https://opentelemetry.io/docs/specs/otlp/?utm_source=chatgpt.com

22 25 GitHub - anchore/syft: CLI tool and library for generating a Software Bill of Materials from container images and filesystems

https://github.com/anchore/syft?utm_source=chatgpt.com

24 38 56 Decision Logs | Open Policy Agent

https://www.openpolicyagent.org/docs/management-decision-logs?utm_source=chatgpt.com

26 36 58 Vulnerability - Trivy

https://trivy.dev/docs/v0.52/guide/scanner/vulnerability/?utm_source=chatgpt.com

27 55 Verifying Signatures - Sigstore

https://docs.sigstore.dev/cosign/verifying/verify/?utm_source=chatgpt.com

28 Automated Sync Policy - Argo CD - Declarative GitOps CD for Kubernetes

https://argo-cd.readthedocs.io/en/latest/user-guide/auto_sync/?utm_source=chatgpt.com

33 35 Timeline for the Implementation of the EU AI Act | AI Act Service Desk

https://ai-act-service-desk.ec.europa.eu/en/ai-act/eu-ai-act-implementation-timeline?utm_source=chatgpt.com

34 Learning environment | Kubernetes

https://kubernetes.io/docs/setup/learning-environment/?utm_source=chatgpt.com

37 40 43 46 53 Pricing | OpenAI API

https://platform.openai.com/docs/pricing/?utm_source=chatgpt.com

42 Secure use reference - GitHub Docs

https://docs.github.com/actions/learn-github-actions/security-hardening-for-github-actions?utm_source=chatgpt.com

45 ISO/IEC 42001:2023 - AI management systems

https://www.iso.org/standard/42001?utm_source=chatgpt.com

48 50 54 NIST AI 100-1

https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf?utm_source=chatgpt.com